

Introduction to Git and Version Control

TechTeam 10/23/2025
Workshop

What is Git? Why is it helpful?

Some questions we can think about:

- 1.) What is important when working on code with other people?
- 2.) What are possible issues that could arise?
- 3.) What are things you want to keep track of?

Hint: think about the steps of sharing a doc and writing things with other people at the same time

Version Control`

- Keeps a history of file changes
- Allows collaboration without overwriting work
- Enables you to roll back to previous documents

Github allows for users to have version control with their code to collaborate effectively and safely

- Use branches to branch off code and safely experiment before “pushing” back to the main branch

Key Terms

- **Repository (repo):** project folder tracked by Git
- **Commit:** A snapshot of your project at a moment in time
- **Branch:** An independent line of development
- **Merge:** combine changes from different branches
- **Remote:** a version of your repository stored online (e.g. GitHub: allows users to store their repositories)

Let's Try It!

-If you haven't made a GitHub account, do it now-

Installing Git:

Windows: Git for Windows

Mac: *brew install git*

Linux: *sudo apt install git*

To check if its installed, check: *git --version*

Setting up SSH and connecting Git to Github

- git needs to know what GitHub account to use!
- Skip this if you already have git configured

Steps (type these in carefully!):

1. Open “git bash” and type the following commands:
 1. `git config --global user.name “First Last”`
 2. `git config --global user.email “youremail@uchicago.edu”`
 - a. This must be the email you used to create your account!

Try running `git config --global user.email` to verify!

SSH keys

- Again, if you've already done this, skip!

1. Make an ssh key with `ssh-keygen`
 - a. Hit "enter" to skip destination/password
 - b. If it says you already have one, use that!
2. Go to <https://github.com/settings/keys>
 - a. Click "New ssh key" and set some title
 - b. Open ".ssh/id_rsa.pub" (use cat on linux, notepad or another text editor on windows/mac)
 - c. Copy ALL TEXT (CTRL-A) into "key"

```
Microsoft Windows [Version 10.0.22621.2283]
(c) Microsoft Corporation. All rights reserved.

C:\Users\basic>ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (C:\Users\basic\.ssh\id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\basic\.ssh\id_rsa
Your public key has been saved in C:\Users\basic\.ssh\id_rsa.pub
The key fingerprint is:
SHA256:izj1...QV/Gc basic@B00K-OP2M7N5I01
The key's randomart image is:
+---[RSA 3072]-----+
|  ==*o+. . o |
| 0==o o . . = . E |
| +oo o . S = . o |
|  |  |  |  |  | |
+---[SHA256]-----+

C:\Users\basic>
```

Add new SSH Key

Title

Key type

Key

```
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQgQCCh9WeFrcSGsj3vhLuhRnCV05yxtMvoAK9oayGypX5y8hQgb
KpeQQkpbSisbpLHU3KN5omJ9Ra5ekhDIHIVQfBVeftb8oMJCxXaBa42wpXkF2TW89L6380Rwq7VW
Ch2ZJQnRbEmOhMhtirtPWEzZ+OjKbJq4SXcOypvlatun0QfzRuDsargiu+A4s= basic@B00K-
OP2M7N5I01
```

Add SSH key

Your First Repository

Go to terminal (Type these commands after the \$)

```
$ mkdir my-first-repo  
$ cd my-first-repo  
$ git init
```

This creates a git repo! (replace *my-first-repo* with any name you'd like)

Command breakdown

mkdir -> (linux) make directory

cd -> (linux) change directory

git init -> initialize (git command, foundation before you can use other git commands)

Tracking Changes

Let's make a file and commit a change!

```
$ touch file.txt
```

```
$ git add file.txt
```

```
$ git commit -m "Add file.txt"
```

Now Git knows what's changed!

Command breakdown

touch -> (linux) makes a new, empty file

git add -> specifies changes you want to include in your commit

git commit -> records changes in the local Git repository

- When you are coding you want to commit often to save those changes :)

Viewing History

```
$ git log
```

```
$ git status
```

See what's new and what's **staged** (what has been added but not committed yet)

Command breakdown

git log -> shows you your commit history

git status -> shows you the state of your Git working directory and staging area. This includes modified files, untracked changes, or staged changes

Connecting to GitHub

```
$ git remote add origin https://github.com/username/repo.git
```

git remote -> manages connection to other repo

add -> combined with add here creates a new remote connection

origin -> the alias Git uses to refer to the remote repository

https://github.com/username/repo.git -> the URL of your remote repo on GitHub

```
$ git push -u origin main
```

git push -> upload your commits to a remote repo

origin -> The remote you just added (the GitHub repo)

main -> The branch name you're pushing to, usually is the main branch

-u -> helps for future, sets this remote branch as the “upstream” meaning future commands will default here

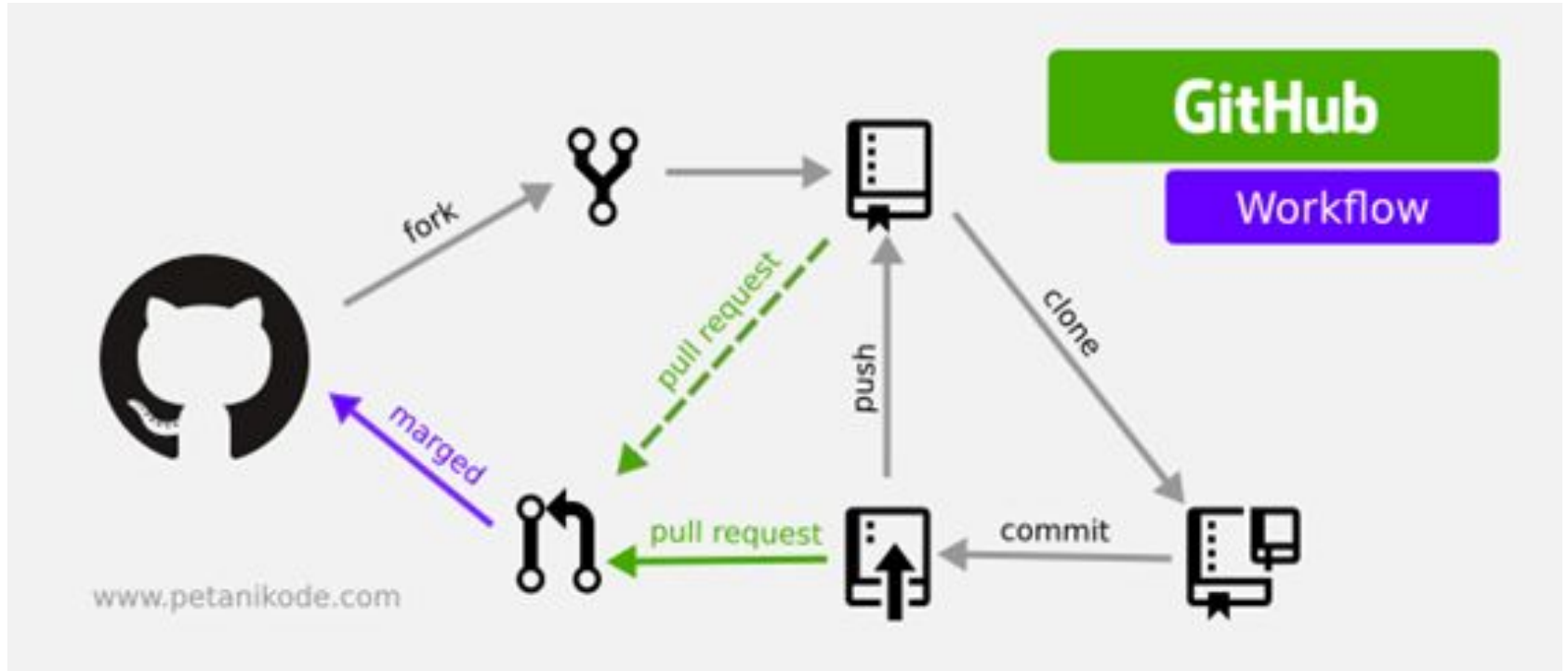
Branching Basics

\$ git branch *feature_1* -> creates new branch called *feature_1*, name it whatever you want but it's good practice to name it after the feature/code you're implementing

\$ git checkout *feature_1* -> switches to the new branch (don't forget! You don't want to be coding and making changes to the wrong branch)

\$ git merge *feature_1* -> after you're done making your changes, you can merge this branch to main

Workflow Overview



Fork: independent copy on GitHub

Clone: local copy on your computer



Questions?



Thanks for listening!

If you think of questions later, reach out to zaynacheema@uchicago.edu !